Project Documentation

1. Overview

DyslexiaDetectionSystem is a multimodal learning-disorder screening and support platform. It is built around educational signals rather than a single medical model, and it is designed to work well in low-resource, multilingual, and local/offline-friendly settings. Dyslexia is the primary use case, but the documentation now treats it as part of a broader learning-disorder workflow.

The repository currently contains three main application surfaces:

1. A Streamlit dashboard in app.py (/d:/Project/DyslexiaDetectionSystem/app.py)
1. A standalone local browser dashboard in web/ (/d:/Project/DyslexiaDetectionSystem/web)
1. A React/Vite frontend in frontend/ (/d:/Project/DyslexiaDetectionSystem/frontend)

The project combines:

- handwriting image analysis
- reading-audio feature extraction
- multilingual text encoding
- reading-behavior observations
- speech-therapy scoring and planning
- visual-focus / eye-tracking analysis
- digital biomarker discovery
- explainability and educational feedback
- record keeping and final report generation

This system is a screening and assistance tool. It is not a clinical diagnostic product.

2. How The Project Is Organized

The repository is easiest to understand as five layers:

1. src/dyslexia_detection/

Core Python package containing models, preprocessing, explainability, training, intervention, biomarker discovery, eye tracking, and utility logic.

1. Dashboards

User-facing surfaces for running the workflows interactively.

1. scripts/

Command-line helpers for dataset preparation, training, transfer learning, optimization, and analysis.

1. data/, checkpoints/, reports/, exports/

Storage locations for manifests, trained weights, analysis outputs, and exported artifacts.

1. web/assets/

Static browser assets, including audio prompts for the standalone local dashboard.

3. Repository Map

Root-level files

• app.py (/d:/Project/DyslexiaDetectionSystem/app.py)

Main Streamlit dashboard and orchestration layer.

• prototypeapp.py (/d:/Project/DyslexiaDetectionSystem/prototypeapp.py)

Earlier prototype-oriented Streamlit interface.

• runlocalweb.py (/d:/Project/DyslexiaDetectionSystem/runlocalweb.py)

Local HTTP server for the standalone browser dashboard and microphone-enabled audio transcription.

• README.md (/d:/Project/DyslexiaDetectionSystem/README.md)

Quick project summary and usage notes.

• requirements.txt (/d:/Project/DyslexiaDetectionSystem/requirements.txt)

Python dependency list.

Python package

• src/dyslexiadetection/ (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection)

Core machine-learning and educational logic.

Web dashboard

• web/index.html (/d:/Project/DyslexiaDetectionSystem/web/index.html)

Dashboard structure and visible controls.

• web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js)

Browser-side workflow logic, scoring, records, and PDF export.

• web/styles.css (/d:/Project/DyslexiaDetectionSystem/web/styles.css)

Dashboard styling.

• web/assets/audio/ (/d:/Project/DyslexiaDetectionSystem/web/assets/audio)

Local audio prompts used by the reading/listening flows.

React frontend

• frontend/src/App.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/App.jsx)

React dashboard UI.

• frontend/src/main.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/main.jsx)

React entry point.

• frontend/src/styles.css (/d:/Project/DyslexiaDetectionSystem/frontend/src/styles.css)

React dashboard styling.

Documentation

• docs/PROJECTDOCUMENTATION.md
(/d:/Project/DyslexiaDetectionSystem/docs/PROJECTDOCUMENTATION.md)

Detailed repository documentation.

• docs/DATASETS.md (/d:/Project/DyslexiaDetectionSystem/docs/DATASETS.md)

Full dataset guide covering demo, collection, benchmark, and biomarker test assets.

• docs/ARCHITECTURE.md (/d:/Project/DyslexiaDetectionSystem/docs/ARCHITECTURE.md)

System architecture, component flow, model families, and runtime design.

• docs/MODELCATALOG.md (/d:/Project/DyslexiaDetectionSystem/docs/MODELCATALOG.md)

Detailed catalog of all used models, their purpose, strengths, gaps, and input requirements.

• docs/PROJECTSCOPES.md (/d:/Project/DyslexiaDetectionSystem/docs/PROJECTSCOPES.md)

Detailed scope breakdown and research directions for the project.

• docs/REFERENCES.md (/d:/Project/DyslexiaDetectionSystem/docs/REFERENCES.md)

Curated paper links and reference list for the project.

• docs/RESEARCHPAPERDRAFT.md
(/d:/Project/DyslexiaDetectionSystem/docs/RESEARCHPAPERDRAFT.md)

Paper-style draft based on the current codebase and architecture.

• docs/DyslexiaDetectionSystemDemoDeck.pptx
(/d:/Project/DyslexiaDetectionSystem/docs/DyslexiaDetectionSystemDemoDeck.pptx)

28-slide demo presentation for walkthroughs and demonstrations.

• docs/researchproposals/ (/d:/Project/DyslexiaDetectionSystem/docs/researchproposals)

Optional proposal notes for multimodal screening, low-resource transfer, and explainable intervention research.

• docs/furtherresearch/ (/d:/Project/DyslexiaDetectionSystem/docs/furtherresearch)

Research roadmap, data needs, experiment matrix, and publication directions.

• docs/screenshots/ (/d:/Project/DyslexiaDetectionSystem/docs/screenshots)

Reserved dashboard screenshot folder for future captures.

3.1 Model comparison snapshots

The current docs and dashboard flows use a snapshot-driven comparison layer rather than a single live score. That keeps validation summaries, model rankings, and holdout checks easier to interpret.

In the standalone web dashboard, some model-statistics panels use display-only label swapping for vit and transformer, so the visible labels may differ from the underlying model identifiers described here.

Useful reference files:

• README.md (/d:/Project/DyslexiaDetectionSystem/README.md)

• docs/MODELCATALOG.md (/d:/Project/DyslexiaDetectionSystem/docs/MODELCATALOG.md)

• docs/furtherresearch/experimentmatrix.md (/d:/Project/DyslexiaDetectionSystem/docs/furtherresearch/experimentmatrix.md)

Current ranking order used in the latest comparison snapshot:

1. multimodal_attention
1. transformer
1. vit

4. User-Facing Surfaces

4.1 Streamlit dashboard

Primary file:

• app.py (/d:/Project/DyslexiaDetectionSystem/app.py)

Purpose:

• dataset overview and analysis
• sample collection
• live screening
• webcam screening
• biomarker discovery
• speech therapy support
• eye-tracking metrics
• intervention recommendations
• final report generation
• federated training and deployment utilities

This is the most feature-complete Python surface.

4.2 Standalone local browser dashboard

Primary files:

• web/index.html (/d:/Project/DyslexiaDetectionSystem/web/index.html)
• web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js)
• web/styles.css (/d:/Project/DyslexiaDetectionSystem/web/styles.css)

Purpose:

- browser-first local workflow
- screening, therapy, visual focus, biomarkers, and records in one page
- local PDF export
- local history storage
- microphone-enabled reading and speech flows

This surface is launched from runlocalweb.py (/d:/Project/DyslexiaDetectionSystem/runlocalweb.py), which serves the files from web/ on localhost:8080.

4.3 React frontend

Primary files:

- frontend/src/App.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/App.jsx)
- frontend/src/main.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/main.jsx)

Purpose:

- a Vite-based React interface
- API-connected dashboard views
- report composition and export support

The React frontend is separate from the standalone browser dashboard.

5. Core Package Modules

5.1 Configuration and shared schemas

- config.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/config.py)

Defines data configuration, training defaults, and language character sets.

- schemas.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/schemas.py)

Defines the shared manifest schema, behavior columns, eye-tracking columns, ethics fields, and collection metadata fields.

Working principle:

- DataConfig keeps image size, sample rate, audio frame count, and text length consistent across the codebase.
- Language-specific character sets determine the text vocabulary used by text encoders.
- schemas.py acts as the contract for what the manifest and collection pipeline expect.

5.2 Data loading and preprocessing

• dataset.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/dataset.py)

Manifest-based PyTorch dataset that resolves file paths and returns tensors.

• preprocessing.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/preprocessing.py)

Image normalization, audio feature extraction, and text encoding.

Working principle:

• handwriting images are loaded as grayscale, contrast-normalized, padded to a fixed canvas, and converted into 1 x H x W tensors

• audio files are read as mono WAV, optionally resampled, transformed into a log-spectrogram-like feature map, and normalized

• text is NFC-normalized, optionally lowercased for English/Latin content, tokenized at character level, and padded to a fixed length

• behavior features are kept numeric and passed through as a small vector

5.3 Model definitions

• models.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/models.py)

Contains the model families used throughout the repository.

Working principle:

• separate encoders learn modality-specific representations

• the encoders are fused into a shared latent space

• a classifier head produces risk, severity, or regression outputs depending on the task

• an attention variant can assign explicit modality weights

5.4 Main training loop

• train.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/train.py)

Generic training entry point for the main multimodal architectures.

Working principle:

• loads the CSV manifest into a dataset

• splits into train and validation subsets

• builds the requested architecture

• supports binary risk, 3-level severity, or regression

• optionally initializes from an SSL audio checkpoint

• optionally transfers weights from a source-language checkpoint

• optionally uses teacher-guided feature distillation

• saves the best checkpoint and a CSV training history

5.5 Severity helpers

• severity.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/severity.py)

Working principle:

• converts raw error/time signals into a normalized severity score
• maps that score to mild, moderate, or severe labels
• respects explicit severity values if they already exist in the manifest

5.6 Cross-lingual transfer

• crosslingual.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/cross_lingual.py)

Working principle:

• copies matching tensors from a source checkpoint into a target model
• can freeze specific prefixes during warm-up
• computes shared-feature distillation loss between teacher and student models

5.7 Foundation model support

• foundation.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/foundation.py)

Working principle:

• uses handwriting, audio, text, and behavior encoders together
• projects each modality into a common embedding dimension
• normalizes modality embeddings before fusion
• adds masked-text reconstruction and modality contrastive objectives
• provides an adapter head for dyslexia, dysgraphia, or dyscalculia tasks

5.8 Self-supervised audio pretraining

• sslpretraining.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/ssl_pretraining.py)

Working principle:

• learns audio representations without requiring full supervised labels
• supports contrastive learning, masked reconstruction, and teacher-distillation modes
• uses time masking and frequency masking as augmentation

• can initialize the audio branch of downstream multimodal models

5.9 Biomarker discovery

• biomarkers.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/biomarkers.py)

Working principle:

• extracts handcrafted handwriting, speech, and reading-behavior features

• builds a biomarker dataset from a manifest

• estimates feature importance by combining effect size, label correlation, and a logistic regression signal

• produces a ranked summary of candidate biomarkers

5.10 Eye tracking and gaze metrics

• eyetracking.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/eye_tracking.py)

Working principle:

• takes a gaze trace table with timestamps and gaze coordinates

• computes fixation duration, regressions, reading speed, gaze dispersion, scanpath length, and mean saccade velocity

• appends the resulting metrics to a CSV log

5.11 Speech therapy support

• speechtherapy.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/speech_therapy.py)

Working principle:

• defines language-specific therapy tasks

• scores a therapy session from duration, pronunciation errors, repetition, substitution, and attention rating

• writes therapy session logs to CSV

• provides structured task lists for Bengali and English

5.12 Personalized intervention engine

• intervention.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/intervention.py)

Working principle:

• builds an InterventionProfile from the learner's error and fluency pattern

• uses a simple Q-table policy to select one of several intervention actions
• generates reading, pronunciation, and spelling exercises with weekly targets
• updates the policy based on reward from progress

5.13 Adaptive tutoring

• adaptivetutoring.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/adaptive_tutoring.py)

Working principle:

• converts observed fluency and error counts into a discrete tutor state
• selects actions using an epsilon-greedy policy
• updates a Q-table from reward feedback
• logs tutoring events for later inspection

5.14 Explainability

• explainability.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/explainability.py)

Working principle:

• GradCAM creates image heatmaps from a target convolution layer
• vitpatchattention_heatmap approximates patch importance in ViT-style handwriting models
• transformertextattention_scores extracts token-level attention scores from the first transformer layer

5.15 Educational explanations

• educationalexplanations.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/educational_explanations.py)

Working principle:

• converts model output into teacher, parent, and student friendly language
• explains the evidence focus and next steps in plain language
• tailors the explanation to confidence, risk band, and modality attention

5.16 Architecture walkthrough

• architecture.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/architecture.py)

Working principle:

• traces a sample through input, preprocessing, feature extraction, sequence modeling, and classification

• is useful for debugging and for understanding how the multimodal pipeline is assembled

5.17 Optimization and deployment

• optimization.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/optimization.py)
• federated.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/federated.py)

Working principle:

• pruning removes less useful weights from linear and convolutional layers
• dynamic quantization compresses supported layers for faster CPU inference
• TorchScript export creates portable deployment artifacts
• federated training aggregates client updates into a global model and evaluates the merged weights

6. Model Catalog

This repository contains multiple model families, each designed for a slightly different trade-off. The current primary screening trio is:

• AttentionMultimodalModel
• TransformerMultimodalModel
• ViTMultimodalModel

Legacy baselines remain in the codebase for experimentation and historical comparison, but they are no longer the primary ranked set in the dashboard or main documentation.

6.1 Initial CNN model

Class:

• InitialCNNModel in models.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/models.py)

Inputs:

• handwriting image
• reading audio
• spelling/pronunciation error counts

Working principle:

• uses the handwriting encoder and audio encoder
• concatenates those embeddings with the error vector
• sends the result to a small classifier head

Best suited for:

- historical comparison
- compact baseline experiments

6.2 Initial LSTM model

Class:

- InitialLSTMModel

Inputs:

- text sequence
- behavior features
- spelling/pronunciation error counts

Working principle:

- uses a bidirectional LSTM for text
- encodes reading behavior with a small MLP
- concatenates sequence and behavior features before classification

Best suited for:

- historical comparison
- text-centered baseline experiments

6.3 CNN-LSTM model

Class:

- InitialCNNLSTMModel

Inputs:

- handwriting image
- reading audio
- text
- error counts
- behavior vector

Working principle:

• combines the CNN handwriting encoder, audio encoder, LSTM text encoder, and behavior encoder
• concatenates all modality embeddings plus the error features
• classifies using a shallow dense head

Best suited for:

• historical comparison
• early multimodal experiments

6.4 Multimodal model

Class:

• MultimodalDyslexiaModel

Working principle:

• uses the CNN handwriting encoder
• uses the audio encoder
• uses a bidirectional GRU-based text encoder
• uses a behavior encoder
• fuses the four modality embeddings and the error vector

Best suited for:

• the default screening pipeline
• balanced multimodal inference

6.5 Transformer multimodal model

Class:

• TransformerMultimodalModel

Working principle:

• same basic multimodal setup as the default model
• replaces the GRU text encoder with a transformer encoder
• uses learned positional embeddings and masked pooling

Best suited for:

• tasks where text order and token context matter more

6.6 ViT multimodal model

Class:

• ViTMultimodalModel

Working principle:

• replaces the CNN handwriting encoder with a patch-based vision transformer encoder
• keeps audio, text, and behavior branches

Best suited for:

• handwriting tasks where patch-level attention is useful

6.7 ViT + Transformer multimodal model

Class:

• ViTTransformerMultimodalModel

Working principle:

• uses a vision transformer for handwriting
• uses a transformer for text
• keeps audio and behavior branches

Best suited for:

• experiments with stronger sequence modeling in both image and text branches

6.8 Attention multimodal model

Class:

• AttentionMultimodalModel

Working principle:

• projects all modality features to a shared dimension
• computes a learned attention score per modality
• forms a weighted fused representation
• exposes the latest modality-attention weights in lastmodalityattention

Best suited for:

• interpretation and modality importance analysis

6.9 Foundation model

Class:

• BengaliLearningDisorderFoundationModel in foundation.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/foundation.py)

Working principle:

• all four modalities are encoded
• each modality is projected into a shared embedding space
• embeddings are normalized
• masked text reconstruction and modality contrastive learning encourage general representations
• reconstructed behavior/error heads encourage the latent space to retain useful task information

Best suited for:

• transfer learning
• low-resource adaptation
• multi-disorder reuse

6.10 Adapter head

Class:

• LearningDisorderAdapter

Working principle:

• reuses a pretrained foundation model
• attaches a small task head for dyslexia, dysgraphia, or dyscalculia

Best suited for:

• disorder-specific fine-tuning

6.11 SSL audio models

Classes in sslpretraining.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/ssl_pretraining.py):

• AudioMaskedReconstructionModel

• AudioContrastiveModel
• AudioTeacherDistillModel

Working principle:

• audio features are learned without a full supervised label set
• contrastive mode learns invariance between augmented views
• masked mode reconstructs masked spectrogram regions
• teacher-distill mode aligns a student embedding with a teacher signal

Best suited for:

• initializing audio encoders before multimodal training

7. Data Flow

The central data flow is consistent across the repository.

7.1 Manifest-driven sample loading

Typical manifest columns are defined in schemas.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/schemas.py).

Core fields:

• sample_id
• student_hash
• handwriting_path
• audio_path
• text_sample
• spelling_errors
• pronunciation_errors
• label

Behavior fields:

• readingtimeseconds
• hesitation_count
• repetition_count
• omission_count

7.2 Preprocessing sequence

1. Resolve file paths relative to the manifest location.
1. Load handwriting as normalized grayscale.
1. Convert audio to a fixed-size spectral representation.
1. Normalize and encode text at character level.
1. Collect error counts and behavior counts as numeric tensors.
1. Feed all of the above into the chosen model.

7.3 Prediction sequence

1. Encoders produce modality-specific embeddings.
1. Fusion layers combine the embeddings.
1. The classifier head outputs logits or a regression score.
1. Softmax or score conversion creates the user-facing result.
1. Explainability and educational text turn the model output into a more readable summary.

8. Dashboard Components

8.1 Standalone web dashboard sections

The browser dashboard in web/index.html (/d:/Project/DyslexiaDetectionSystem/web/index.html) and web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js) currently contains:

- Screening
- Speech Therapy
- Visual Focus Test
- Test Lab & Report
- Biomarkers
- Records

How each section works:

- Screening combines reading fluency, audio listening, and spelling scoring into one screening workflow.
- Speech Therapy runs a guided round, captures microphone input, and scores the response automatically.
- Visual Focus Test runs a symbol-matching task with round-based scoring and saved results.
- Test Lab & Report compares saved results and builds the final report.
- Biomarkers analyzes uploaded CSV data and ranks the strongest signals.
- Records stores local sessions, supports filtering, and shows record details.

Important browser-side behavior:

• microphone access requires localhost or HTTPS

• local records are persisted in browser storage

• the local server can transcribe reading audio using Whisper through the /api/reading-transcribe endpoint in runlocalweb.py (/d:/Project/DyslexiaDetectionSystem/runlocalweb.py)

• the digit mode in Visual Focus Test is a matching task, not handwritten digit classification

8.2 Streamlit dashboard sections

The Streamlit app in app.py (/d:/Project/DyslexiaDetectionSystem/app.py) exposes a broader research workflow.

Current tab groups include:

• Biomarkers

• Dataset Creation

• Sample Collection

• Live Screening

• Webcam Screening

• Speech Therapy

• Eye Tracking

• Final Report

• Federated Training

• Lightweight Deployment

• Foundation Model

It also includes explanation tabs for teacher, parent, and student views.

8.3 React dashboard

The React frontend in frontend/src/App.jsx (/d:/Project/DyslexiaDetectionSystem/frontend/src/App.jsx) provides:

• tabbed screening views

• report composition

• chart visualizations

• API-backed workflows

9. Training, Analysis, and Deployment Workflows

9.1 Training

The main training entry point is src/dyslexiadetection/train.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/train.py).

It supports:

- binary risk prediction
- severity classification
- regression-style severity scoring
- the current three-model screening trio plus legacy CNN/LSTM baselines for comparison

9.2 Cross-lingual transfer

Cross-lingual workflows use crosslingual.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/cross_lingual.py) and related scripts.

Typical idea:

- start from a source-language checkpoint
- copy matching feature extractor weights
- optionally freeze transferred branches
- fine-tune on the target language
- optionally add feature-level distillation

9.3 Foundation and SSL workflows

Foundation-model-style training and audio SSL pretraining are designed to help when supervised data is limited.

Typical idea:

- learn reusable encoders first
- adapt them to a specific task later
- reuse the learned representation for related learning-disorder tasks

9.4 Optimization

optimization.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/optimization.py) supports:

- weight pruning
- dynamic quantization
- TorchScript export
- simple latency benchmarking

These steps are intended for compact local deployment and faster CPU inference.

9.5 Federated training

federated.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexia_detection/federated.py) supports a simple federated-learning style loop.

Working principle:

- local clients train on their own manifests
- model weights are aggregated centrally
- the merged model is evaluated on a validation manifest

This is useful for privacy-sensitive or institution-separated training settings.

10. Persistence and Outputs

The project writes results to different places depending on the surface:

- CSV manifests for training and evaluation
- checkpoint folders for model weights
- CSV logs for therapy, tutoring, intervention, and eye-tracking records
- local browser storage for standalone web records
- report folders for biomarker summaries and exported documents
- PDF output from the final report workflows

Common output locations in the repository include:

- checkpoints/ (/d:/Project/DyslexiaDetectionSystem/checkpoints)
- reports/ (/d:/Project/DyslexiaDetectionSystem/reports)
- exports/ (/d:/Project/DyslexiaDetectionSystem/exports)
- data/ (/d:/Project/DyslexiaDetectionSystem/data)

11. Important Working Principles

11.1 Multimodal design

The system does not rely on a single feature source. It combines handwriting, audio, text, behavior, and error observations so one weak signal does not dominate the result.

11.2 Language awareness

The project supports Bengali, English, and multilingual workflows. Character vocabularies and UI labels are language-sensitive, and the dashboards can switch language for display text and prompts.

11.3 Explainability first

Model output is converted into:

- modality importance
- heatmaps
- attention scores
- teacher/parent/student explanations

That design helps make the system more useful in educational settings.

11.4 Local-first operation

The browser dashboard is designed to work locally. The local launcher exists because microphone capture and browser APIs need a secure local origin.

11.5 Not clinical diagnosis

The outputs are for screening support and educational planning. They should be reviewed alongside teacher, parent, or specialist observation.

12. Recommended Entry Points

If you want to understand the project quickly, start here:

1. src/dyslexiadetection/config.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/config.py)
1. src/dyslexiadetection/preprocessing.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/preprocessing.py)
1. src/dyslexiadetection/models.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)
1. src/dyslexiadetection/train.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/train.py)
1. src/dyslexiadetection/architecture.py (/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/architecture.py)
1. web/app.js (/d:/Project/DyslexiaDetectionSystem/web/app.js)
1. app.py (/d:/Project/DyslexiaDetectionSystem/app.py)

13. Documentation Notes

If you rename a dashboard tab, add a model, change a manifest field, or alter the report flow, update this file at the same time. The repository has multiple UIs, so a change in one surface does not

automatically appear in the others.

14. Summary

This project is a multimodal learning-disorder screening and support ecosystem with:

- reusable Python model and data-processing code
- a Streamlit research dashboard
- a standalone browser dashboard
- a React frontend
- local records and report generation
- explainability and intervention planning

The codebase is intentionally broad because it supports both research exploration and practical local workflows.